

인공지능과



파이선 프로그래밍 기초

Email : sooyeong.l@theastrallab.com

Tel: 042-710-0630

Phone : 010-6290-3060

1. 파이썬과 분석 인프라의 이론적 이해

인공지능 에이전트가 가장 잘 부릴 수 있고 현업 데이터 가공 효율이 가장 극대화된 언어가 바로 **파이썬(Python)**입니다. 프로그래밍을 시작하기 전 알아야 할 핵심 전산 지식입니다.

① 파이썬(Python)이란 무엇인가?

1991년 귀도 반 로섬이 발표한 프로그래밍 언어로, "**가독성이 높고 간결한 문법 구조**"를 지향합니다. 사람이 쓰는 영어 언어 장표와 가장 유사하게 설계되어 있어 학습 장벽이 매우 낮으며, 전 세계 수많은 개발자 및 데이터 분석가들이 구축해 둔 오픈소스 생태계가 가장 거대하게 형성되어 있는 범용 소프트웨어 언어입니다.

② 컴파일 언어 vs 인터프리터 언어의 원리와 특징

컴퓨터 하드웨어(CPU)는 오직 0과 1로 이루어진 이진 기계어만 읽을 수 있습니다. 사람이 영어 기조로 쓴 소스 코드를 기계어로 통역 번역하는 방식에 따라 컴퓨터 언어는 두 종류로 갈립니다.

구분	컴파일 언어 (Compile Language)	인터프리터 언어 (Interpreter Language)
번역 방식	소스 코드 전체를 기동 전에 미리 기계어로 일괄 번역 하여 실행 파일 (.exe 등)을 제작함.	프로그램을 실행하면서 코드를 한 줄씩 실시간으로 번역 하여 작동시킴.
대표 언어	C, C++, Java 등	Python , JavaScript, R 등
장점	이미 일괄 번역이 완료되어 있으므로 컴퓨터의 실행 속도가 극도로 빠름 .	빌드(컴파일) 대기 시간 없이 코드를 작성하는 즉시 실행 결과를 보며 빠르게 수정 가능
단점	코드에 오타가 단 하나만 있어도 컴파일 자체가 거부되며, 디버깅과 빌드 과정이 다소 무거움.	실행 중에 한 줄씩 번역 하므로 실행 성능 속도가 컴파일 언어 대비 상대적으로 조금 느림.

2. 파이썬과 분석 인프라의 이론적 이해

③ IPython(Interactive Python)은 무엇인가?

기본 파이썬 실행기는 코드를 작성하고 실행하면 결과를 본 뒤 바로 프로세스가 닫혀버리는 단발성 구조를 띠니다. 이를 보완해 개발된 **IPython**은 실행 콘솔 창을 열어둔 상태에서 메모리 상에 변수를 계속 띄워놓고 **"대화하듯이(Interactive) 한 줄 실행 → 결과 확인 → 다음 줄 실행"**을 이어나가게 해주는 대화형 확장 실행기입니다. 우리가 실습에 사용할 **주피터 노트북(.ipynb)**이 바로 이 IPython 엔진을 기반으로 구현된 대표적인 대화형 실습 웹 애플리케이션입니다.

④ pip(Python Package Installer)란 무엇인가?

파이썬의 가장 강력한 무기는 전 세계 데이터 분석가들이 미리 짜놓은 엑셀 가공 도구(Pandas), 시각화 도구(Matplotlib), 대시보드 도구(Streamlit) 등의 유틸리티 패키지가 무료 배포된다는 점입니다. **pip**는 이 거대한 파이썬 외부 도서관(PyPI 서버)에 접속하여, 터미널 명령어 한 줄(pip install 패키지명)로 외부 도구들을 내 가상환경 방으로 안전하게 배송 및 자동 설치해 주는 **파이썬 전용 패키지 관리 매니저**입니다.

3. 프롬프트 기반 코딩 가이드

1일차 기본 파이썬 문법 실습은 복잡한 영문 코드를 직접 타이핑해 짜는 과거의 하드코딩 방식이 아닙니다.
교재에 수록된 한글 프롬프트를 활용해 생성형 AI가 주는 코드를 가동하는 **프롬프트 기반의 실습 파이프라인**을 따라갑니다.

프롬프트-코드 연동 실습 4단계 파이프라인

01

프롬프트 복사

각 예제마다 수록된 **2. 예제 프롬프트** 박스 내부의 정돈된 한글 지시문을 그대로 복사합니다.

03

코드 블록 복사

Gemini가 실시간으로 분석해 사출해 준 파이썬 소스 코드 블록의 우측 상단 Copy 버튼을 눌러 코드를 복사합니다.

02

AI(Gemini 등) 질의

복사한 한글 지시문을 생성형 AI(Gemini 등)의 채팅창에 붙여넣어 실행을 요청합니다.

04

주피터 이식 및 실행

실행 상태인 **주피터 노트북(.ipynb) 실습 파일의 빈 코드 셀에 그대로 붙여넣기(Ctrl+V)**하고 실행(Shift + Enter)하여 결과가 올바르게 인쇄되는지 눈으로 확인합니다.

3. 프롬프트 기반 코딩 가이드

원하는 수자원 분석 결과 코드를 AI로부터 오작동 없이 한 번에 얻어내기 위해 작성하는 프롬프트는 반드시 다음 **3대 뼈대 요소**를 갖추어야 합니다. 본 과정의 예제 프롬프트들 역시 이 공식을 기반으로 설계되어 있습니다.

원칙	구성 요소	설명 및 실무 정의	수자원 분석 프롬프트 작성 예시
1. 맥락 (Context)	AI가 상황에 맞는 라이브러리를 선택할 수 있도록 역할 및 배경 상황 을 인지시킵니다.	"너는 K-water의 정수장 약품 투입 관제관이야. 실시간 원수 탁도 및 약품 잔량 엑셀 원장이 주어졌어."	
2. Task (작업)	AI가 수행해야 할 구체적인 전처리 및 연산 작업 을 동사형 명령으로 명확히 지시합니다.	"엑셀 파일에서 탁도가 0.5 NTU를 초과해 위험한 정수 구역들만 Pandas로 골라내는 필터링 코드를 작성해 줘."	
3. 제약사항 및 포맷 (Constraints)	코드의 안전 장치(예외 처리), 코드 포맷, 주석 여부 등 출력 규칙 을 엄격히 제한합니다.	"반드시 try-except로 파일 누락 예외를 처리하고, 비전공자도 이해하기 쉬운 한글 주석을 라인마다 기입해 줘."	

환경 구축 가이드

본 과정은 파이썬 배포판 중 경량화되어 가상환경 관리에 특화된 **Miniconda(미니콘다)** 또는 Anaconda 환경이 이미 설치되어 있음을 전제로 실습을 가동합니다. 윈도우(Windows) 환경에서 수자원 데이터 분석에 필요한 라이브러리를 안전하게 격리 구축하고 구동하는 절차입니다.

가상환경 생성 및 활성화 (Conda)

파이썬은 프로젝트별로 라이브러리 버전 충돌을 막기 위해 독립된 방인 **가상환경(Virtual Environment)**을 만들어 사용합니다. 윈도우 시작 메뉴에서 "**Anaconda Command Prompt**" 또는 "**Command Prompt**"을 실행하고 다음 명령어를 순차 입력합니다.

```
# 1. 'kwater_env'라는 이름의 파이썬 3.10 버전 가상환경 생성
conda create -n kwater_env python=3.10 -y
```

```
# 2. 생성된 가상환경 활성화 (활성화 시 프롬프트 맨 앞에 (kwater_env)가 표시됨)
conda activate kwater_env
```

1. 원시 데이터 처리와 문자열 가공

1-1. 변수 할당과 출력

1. 개념 설명 및 동작 원리

변수(Variable)는 데이터를 컴퓨터 메모리(임시 저장 공간)에 저장하고 이름을 붙여 사용하는 가상의 상자입니다. 파이썬에서는 등호(=) 기호를 기준으로 오른쪽에 적힌 데이터 값을 왼쪽에 적힌 변수 이름에 집어넣는 **할당(Assignment)** 작업을 수행합니다. 저장된 데이터를 화면에 출력하여 눈으로 확인하고 싶을 때는 `print()` 함수를 사용합니다.

[문법 구조]

```
변수이름 = 저장할데이터  
print(변수이름)
```

2. 예제 프롬프트

정수장 센서가 수집한 문자열 데이터 " 용담정수장,pH_7.2,양호 "를 변수 `sensor_raw_string`에 저장한 뒤, `print()` 함수를 이용해 이 변수값을 화면에 출력하는 파이썬 코드를 작성해줘. 코드 한 줄마다 초보자도 이해하기 쉬운 간단한 설명 주석을 달아줘.

3. 실행 결과

```
용담정수장,pH_7.2,양호
```

1-2. 문자열 공백 제거 (strip)

1. 개념 설명 및 동작 원리

통신 센서나 외부 파일에서 텍스트 데이터를 읽어올 때 눈에 보이지 않는 앞뒤의 불필요한 공백 문자(스페이스, 탭, 줄 바꿈 등)가 섞여 유입될 수 있습니다. 파이썬의 문자열 객체가 기본 제공하는 `.strip()` 메서드는 문자열 양 끝의 공백만을 제거하여 데이터를 깔끔하게 세척합니다.

[문법 구조]

```
정제된_문자열 = 문자열변수.strip()
```

2. 예제 프롬프트

문자열 변수 `sensor_raw_string`에 " 용담정수장,pH_7.2,양호 "가 들어 있습니다. 문자열의 앞뒤 공백을 지운 깨끗한 문자열을 변수 `clean_string`에 저장하고 출력하는 파이썬 코드를 작성해줘. 공백을 제거하는 함수명과 그 역할을 간단한 주석으로 달아줘.

3. 실행 결과

```
용담정수장,pH_7.2,양호
```

1-3. 문자열 분리 (split)

1. 개념 설명 및 동작 원리

구분자(쉼표, 파이프 기호 등)로 연결되어 뭉쳐 있는 하나의 문자열 데이터를 개별 분석에 활용하려면 의미 단위로 쪼개야 합니다. `.split(구분자)` 메서드는 지정한 구분자를 기준으로 텍스트를 마디마디 잘라 여러 개의 데이터가 나열된 **리스트(List)** 구조로 변환하여 줍니다.

[문법 구조]

```
쪼개진_리스트 = 문자열변수.split('구분기호')
```

2. 예제 프롬프트

변수 `clean_string`에 "용담정수장,7.2,양호"가 저장되어 있습니다. 이 문자열을 쉼표(`,`)를 기준으로 나누어 `water_data_list`라는 변수에 저장하고 출력하는 파이썬 코드를 작성해줘. 분할된 결과가 대괄호 `[]` 형태의 리스트로 출력되는 이유를 주석으로 설명해줘.

3. 실행 결과

```
['용담정수장', '7.2', '양호']
```

1-4. 리스트 인덱싱을 통한 요소 추출

1. 개념 설명 및 동작 원리

리스트(List)는 순서가 있는 데이터들의 나열 바구니입니다. 파이썬에서는 순서 번호인 **인덱스(Index)**가 **0번부터 시작**한다는 독특한 규칙이 있습니다. 리스트변수[인덱스번호] 형식을 사용하여 대괄호 안에 순서 번호를 기입하면, 해당 위치의 데이터 원소 하나만을 콧 집어 참조해 올 수 있습니다.

[문법 구조]

첫번째요소 = 리스트변수[0]
두번째요소 = 리스트변수[1]

2. 예제 프롬프트

리스트 `water_data_list = ['용담정수장', '7.2', '양호']`가 있습니다. 이 리스트에서 0번째 방의 정수장 이름과 1번째 방의 pH 지수를 각각 추출하여 변수 `plant_name`과 `ph_str`에 저장하고 화면에 출력하는 파이썬 코드를 작성해줘. 인덱스가 0부터 시작한다는 원리를 주석으로 달아줘.

3. 실행 결과

정수장 이름: 용담정수장
pH 지수: 7.2

1-5. 데이터 타입 변환 (float)

1. 개념 설명 및 동작 원리

파이썬은 변수에 저장된 값의 성격에 따라 데이터 타입(Data Type)을 매깁니다. 따옴표로 감싸진 "7.2"는 사칙연산이 불가능한 글자 타입(str)이고, 소수점이 붙은 7.2는 연산이 보장되는 실수형 타입(float)입니다. 문자형 수치를 실제 수학 계산에 쓰려면 float() 함수를 씌워 실수형으로 명시적 **형변환(Type Casting)**을 시행해야 합니다.

[문법 구조]

```
변환된실수변수 = float(문자열변수)
```

2. 예제 프롬프트

문자열 변수 `ph_str = "7.2"`가 선언되어 있습니다. 이 텍스트 데이터를 실수형(float) 타입으로 형변환하여 `ph_val` 변수에 저장해줘. 변환 전과 후의 타입을 `type()` 함수로 출력하여 타입이 str에서 float으로 교정되었음을 입증하는 파이썬 코드를 작성해줘.

3. 실행 결과

변환 전 타입: <class 'str'>

변환 후 타입: <class 'float'>

1-6. 산술 연산을 통한 수온 변화 예측

1. 개념 설명 및 동작 원리

숫자형 변수(정수 및 실수)들은 우리가 아는 표준 수학 사칙연산 기호(+ 더하기, - 빼기, * 곱하기, / 나누기)를 그대로 적용하여 계산할 수 있습니다.

[문법 구조]

```
결과변수 = 변수1 + 변수2 # 덧셈 예시
```

2. 예제 프롬프트

실수형 수온 변수 `water_temp = 18.5`가 있습니다. 기온 상승 시나리오에 따라 온도가 1.2도 상승할 것으로 예상됩니다. 기존 수온에 1.2를 더하는 산술 연산을 가하여 예상 수온을 구하고 `predicted_temp` 변수에 저장해 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
예상 수온(도): 19.7
```

1-7. f-string 문자열 포매팅

1. 개념 설명 및 동작 원리

변수 데이터와 일반 안내 문장 텍스트를 자연스럽게 결합하여 출력하기 위해 f-string 포매팅을 사용합니다. 문자열 따옴표 시작 앞에 접두사 f를 적고, 문자열 내부에서 중괄호 {변수명}을 명시하면 변수 내부의 실제 데이터 값이 해당 자리에 쏙 채워집니다.

[문법 구조]

```
f"텍스트 {변수명1} 텍스트 {변수명2}"
```

2. 예제 프롬프트

변수 `plant_name = "용담정수장"`과 실수형 변수 `ph_val = 7.2`가 있습니다. f-string 포매팅을 사용해 "현재 용담정수장의 pH 농도는 7.2입니다."라는 종합 출력 문장을 인쇄하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
현재 용담정수장의 pH 농도는 7.2입니다.
```

1-8. 문자열 치환 (replace)

1. 개념 설명 및 동작 원리

문자열 내의 특정 문자 패스(Substring)를 지목하여 원하는 다른 텍스트로 바꾸고자 할 때 문자열 객체의 `.replace(찾을단어, 바꿀단어)` 메서드를 사용합니다. 원본 변수 자체는 훼손하지 않으면서 변환된 새로운 결과 문자열을 반환합니다.

[문법 구조]

```
치환된문자열 = 원본문자열.replace('기존단어', '신규단어')
```

2. 예제 프롬프트

변수 `report` = "현재 용담정수장의 여과 장치가 정상 가동 중입니다."가 있습니다. 이 텍스트 내에서 "용담정수장"을 "보령정수장"으로 변경하여 `new_report` 변수에 저장하고 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
현재 보령정수장의 여과 장치가 정상 가동 중입니다.
```

1-9. 멤버십 연산자 (in)

1. 개념 설명 및 동작 원리

특정 텍스트가 저장된 변수 내에 수자원 관제 상 확인이 필요한 중요 경보 키워드(예: '위험', '경보', '초과')가 실제로 포함되어 있는지 스캔하고자 할 때 in 연산자를 씁니다. 판정 결과는 True 또는 False로 반환됩니다.

[문법 구조]

포함여부(True/False) = "찾을단어" in 검사할문자열변수

2. 예제 프롬프트

문자열 변수 `status_msg = "염소 저장 탱크 압력 저하: 긴급 경보 발령"`이 주어졌습니다. 이 메시지 내에 "경보"라는 위험 단어가 들어있는지 in 연산자로 검사하여 `alert_status` 변수에 대입하고 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

경보 키워드 존재 여부: True

1-10. 메서드 체이닝을 통한 한 줄 데이터 전처리

1. 개념 설명 및 동작 원리

파이썬은 하나의 대상에 대해 메서드 호출 결과를 곧장 다음 메서드의 입력값으로 넘겨 점(.) 기호로 줄줄이 연동하는 **메서드 체이닝(Method Chaining)**을 지원합니다. 여러 정제 단계를 단 한 행의 코드로 실행할 수 있습니다.

[문법 구조]

```
최종결과 = 변수.메서드1().메서드2().메서드3()
```

2. 예제 프롬프트

미정제 문자열 데이터 `sensor_raw = " 주암정수장|0.15|적합 "`이 주어졌습니다. 메서드 체이닝을 활용해 단 한 줄의 코드로 1. 공백 제거(strip), 2. 파이프(|) 분리(split), 3. 탁도 데이터([1]번 위치) 추출, 4. 실수 변환(float)을 연쇄 수행하여 `clean_turbidity` 변수에 담고 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
최종 탁도 수치: 0.15
```

2. 복합 자료구조 설계: 리스트, 딕셔너리, 튜플

2-1. 리스트 추가 (append)

1. 개념 설명 및 동작 원리

리스트(List)는 파이썬에서 여러 데이터를 하나의 대괄호 [] 묶음 내에 순서대로 관리하는 바구니 자료형입니다. 빈 리스트를 우선 생성해 둔 뒤, 데이터가 수집될 때마다 리스트 객체의 .append(추가할값) 메서드를 호출하면 리스트의 맨 끝자리에 순차적으로 데이터가 삽입됩니다.

[문법 구조]

```
리스트명 = []  
리스트명.append(값1)
```

2. 예제 프롬프트

실시간 탁도 모니터링 데이터를 수집하고자 합니다. 비어 있는 리스트 turbidity_records를 선언한 후, .append() 메서드를 적용해 시간별 수집 탁도 값인 0.12, 0.15, 0.18을 순차적으로 추가하고 리스트의 최종 상태를 확인하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
수집된 탁도 리스트: [0.12, 0.15, 0.18]
```

2-2. 내장 함수를 활용한 리스트 데이터 통계 계산

1. 개념 설명 및 동작 원리

파이썬에는 리스트에 담긴 수치 총합을 구하는 `sum()`, 리스트 내 요소의 총 갯수(데이터 길이)를 반환하는 `len()`, 가장 큰 값을 포착하는 `max()` 등 다양한 내장 함수를 기본 제공합니다. 이들을 조합하면 평균 등을 고속 계산할 수 있습니다.

[문법 구조]

```
합계 = sum(리스트)
갯수 = len(리스트)
평균 = sum(리스트) / len(리스트)
```

2. 예제 프롬프트

5일간의 탁도 관측 기록 리스트 `daily_turbidity = [0.12, 0.22, 0.15, 0.18, 0.13]`이 있습니다. 내장 함수 `sum()`과 `len()`을 연계하여 5일간의 탁도 수치 총합과 평균 탁도를 산출하고 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
탁도 총합: 0.8
평균 탁도: 0.16
```

2-3. 리스트 슬라이싱 (Slicing)

1. 개념 설명 및 동작 원리

슬라이싱(Slicing)은 전체 리스트 데이터 중 지정 구간의 부분집합만을 오려와 새로운 하위 리스트를 부분 복제해 내는 기술입니다. 리스트[시작:끝] 형식을 사용하며, 수학적 경계선 규칙에 의해 지정한 **끝 인덱스 위치는 범위에 포함되지 않고 바로 직전 인덱스까지만** 잘라웁니다.

[문법 구조]

```
부분리스트 = 원본리스트[시작인덱스:끝인덱스] # 끝인덱스는 미포함
```

2. 예제 프롬프트

7일간 관측된 잔류염소 수치 리스트 `weekly_chlorine` = [0.45, 0.48, 0.50, 0.42, 0.40, 0.35, 0.30]이 있습니다. 슬라이싱을 적용해 주말을 제외한 평일 처음 5일간의 데이터(인덱스 0부터 4까지)만 추출하여 `weekday_chlorine` 변수에 대입하고 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
평일 잔류염소 데이터: [0.45, 0.48, 0.50, 0.42, 0.40]
```

2-4. 중첩 리스트와 다차원 인덱싱

1. 개념 설명 및 동작 원리

리스트 내부에 다른 리스트가 요소로 들어가는 구조를 중첩 리스트 또는 2차원 리스트라고 합니다. 엑셀의 행과 열처럼 구조화된 데이터를 표현하며, 리스트명[행번호][열번호] 형태로 대괄호를 두 번 사용하여 특정 격자 공간의 값에 접근합니다.

[문법 구조]

```
2차원원소 = 리스트변수[행인덱스][열인덱스]
```

2. 예제 프롬프트

정수장 이름, 탁도, 잔류염소가 한 세트로 결합된 2차원 리스트

```
plant_data = [['덕소정수장', 0.12, 0.4], ['화도정수장', 0.15, 0.5], ['삼패정수장', 0.18, 0.3]]
```

를 정의해줘. 이 중 두 번째 행에 있는 화도정수장의 잔류염소 수치(0.5)만 다차원 인덱싱으로 추출하여 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
화도정수장 잔류염소: 0.5
```

2-5. 딕셔너리 생성 (Dictionary)

1. 개념 설명 및 동작 원리

딕셔너리(Dictionary)는 데이터를 키(Key)와 값(Value)의 쌍으로 중괄호 {} 내에 매핑하여 저장하는 구조입니다. 리스트처럼 숫자로 조작하는 대신, '이름'이나 '탁도' 같은 명확한 키 단어 이름표를 사용하므로 데이터 조회의 직관성이 높습니다.

[문법 구조]

```
딕셔너리명 = {  
    'Key1': 값1,  
    'Key2': 값2  
}
```

2. 예제 프롬프트

정수장 속성인 이름(Key: 'name', Value: "덕소정수장")과 탁도(Key: 'turbidity', Value: 0.12)를 포함하는 딕셔너리 객체 plant_info를 생성하고 출력하는 파이썬 코드를 작성해줘. 중괄호 {}와 키-값 구분자 콜론 : 사용 규칙에 대한 설명도 주석에 넣어줘.

3. 실행 결과

```
{'name': '덕소정수장', 'turbidity': 0.12}
```

2-6. 딕셔너리 키 삽입 및 수정

1. 개념 설명 및 동작 원리

기존 딕셔너리에 딕셔너리[키] = 새로운값 대입 연산을 명령하면, 만약 기존에 해당 키가 존재했다면 기존 값이 갱신(수정)되며, 존재하지 않던 새로운 키인 경우에는 딕셔너리 구조 내부에 새로운 데이터 쌍이 자동으로 추가됩니다.

[문법 구조]

```
딕셔너리[기존키] = 새로운값 # 수정  
딕셔너리[신규키] = 새로운값 # 추가
```

2. 예제 프롬프트

딕셔너리 `plant_info = {'name': '덕소정수장', 'turbidity': 0.12}`가 선언되어 있습니다. 1. 'turbidity' 키의 값을 0.15로 수정하고, 2. 새로운 키 'operator'에 담당자 이름인 '김철수'를 대입하여 추가한 뒤 최종 딕셔너리를 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
{'name': '덕소정수장', 'turbidity': 0.15, 'operator': '김철수'}
```

2-7. 딕셔너리 안전조회 메서드 (get)

1. 개념 설명 및 동작 원리

딕셔너리에서 대괄호 참조로 값을 꺼내려 할 때, 만약 존재하지 않는 키를 호출하면 에러(KeyError)가 발생해 프로그램 작동이 다운됩니다. `.get(조회키, 기본값)` 메서드는 키가 존재하지 않을 시 에러 대신 미리 정의해 둔 기본 대안 값을 반환하여 안정성을 확보합니다.

[문법 구조]

```
안전한값 = 딕셔너리.get('조회할키', '없을때대체할값')
```

2. 예제 프롬프트

딕셔너리 `plant_info = {'name': '덕소정수장', 'turbidity': 0.15, 'operator': '김철수'}`가 있습니다. 존재하지 않는 잔류염소 농도 키 'chlorine'을 조회하되, 오류 없이 "측정값없음"을 반환하도록 `get()` 메서드를 적용한 파이썬 코드를 작성해줘.

3. 실행 결과

```
잔류염소 농도: 측정값없음
```

2-8. 딕셔너리 키-값 일괄 추출

1. 개념 설명 및 동작 원리

딕셔너리에 매핑 적재되어 있는 모든 속성 이름표(Key) 목록만 긁어보고 싶을 때는 `.keys()` 메서드를 쓰고, 보관 중인 알맹이 데이터(Value) 목록만 모아보고 싶을 때는 `.values()` 메서드를 활용하여 데이터 리스트 형태로 쉽게 변환할 수 있습니다.

[문법 구조]

```
전체키목록 = list(딕셔너리.keys())  
전체값목록 = list(딕셔너리.values())
```

2. 예제 프롬프트

수질 pH 딕셔너리 `ph_dict = {"용담정수장": 7.2, "보령정수장": 6.8, "주암정수장": 7.5}`가 있습니다. 1. `keys()` 메서드로 정수장 이름 목록만 추출하여 리스트로 출력하고, 2. `values()` 메서드로 pH 지수 수치 목록만 추출하여 리스트로 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
등록된 정수장 목록: ['용담정수장', '보령정수장', '주암정수장']  
등록된 pH 수치 목록: [7.2, 6.8, 7.5]
```

2-9. 불변 자료구조 튜플 (Tuple)의 이해

1. 개념 설명 및 동작 원리

튜플(Tuple)은 리스트와 유사하게 여러 데이터를 묶는 시퀀스 자료구조입니다. 하지만 결정적인 이론적 차이는 **"저장된 데이터의 사후 수정 및 변경이 절대 불가능(Immutable)하다"**는 점입니다. 튜플은 소괄호 ()를 사용해 선언하며 데이터 추가(.append)나 변경 대입 시 시스템 오류를 발생시켜 데이터를 강제 보호합니다.

[문법 구조]

```
기준튜플 = (상수값1, 상수값2, 상수값3) # 값 추가/수정/삭제 불가
```

2. 예제 프롬프트

K-water 관리 권역 목록을 안전하게 읽기 전용으로 보관하고자 합니다. "동부", "서부", "남부" 세 개의 문자열을 원소로 가지는 튜플 `kwater_zones`를 생성하고, `print()` 함수로 전체 튜플 내용을 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
('동부', '서부', '남부')
```

2-10. 딕셔너리를 포함한 리스트 인덱싱

1. 개념 설명 및 동작 원리

현업의 데이터베이스 연동 데이터나 웹의 OpenAPI가 반환하는 JSON 구조는 대개 여러 개의 딕셔너리 객체들이 하나의 리스트 바구니 속에 원소로 담겨 들어오는 복합 구조를 보입니다. 이때는 리스트[인덱스][딕셔너리키] 형태로 순차 접근합니다.

[문법 구조]

```
속성값 = 리스트변수[원소순서인덱스]['딕셔너리속성키']
```

2. 예제 프롬프트

중첩 데이터 리스트 db_list = [{'name': '덕소정수장', 'turbidity': 0.12}, {'name': '화도정수장', 'turbidity': 0.15}]가 있습니다. 인덱싱을 적용하여 두 번째 원소(인덱스 1)인 화도정수장의 딕셔너리에 도킹하고, 해당 딕셔너리 속의 탁도 수치(turbidity)만 추출해 출력하는 파이썬 코드를 작성해 줘.

3. 실행 결과

```
화도정수장 탁도: 0.15
```

3. 제어문 (if 조건문, for 반복문)

3-1. 기본 단독 조건문 (if)

1. 개념 설명 및 동작 원리

조건문 if는 제시한 비교 조건식 판정 결과가 참(True)인 경우에만 들여쓰기된 하위 실행 명령 블록을 작동시키는 흐름 제어 구문입니다. 조건문의 끝에는 반드시 콜론(:)을 작성하여 마감하며, 실행 로직은 공백 4칸(탭) 들여쓰기를 적용해야만 파이썬 엔진이 이해합니다.

[문법 구조]

```
if 비교할조건식:  
    조건이참일때_실행할코드
```

2. 예제 프롬프트

실수형 탁도 변수 `turbidity = 0.55`가 선언되어 있습니다. 조건문 if를 활용해 탁도가 기준치인 0.5 NTU 이상일 경우에만 "수질 주의보: 여과막 역세척 필요!"라는 경고를 출력하는 파이썬 코드를 작성해줘. 파이썬의 들여쓰기 문법 규칙을 설명 주석에 담아줘.

3. 실행 결과

수질 주의보: 여과막 역세척 필요!

3-2. 조건 분기문 (if-else)

1. 개념 설명 및 동작 원리

if-else문은 지정한 조건이 성립(True)할 때 가동할 로직 경로와, 성립하지 않는(False) 대안 상황 시 가동될 로직 경로를 정확하게 양분하여 처리하는 상호 배타적 분기 기법입니다.

[문법 구조]

```
if 조건식:
    참일때_가동코드
else:
    거짓일때_대안코드
```

2. 예제 프롬프트

잔류염소 농도 변수 chlorine = 0.15가 있습니다. 염소 수치가 안전선 0.2 이상이면 "수질 안전"을 출력하고, 0.2 미만인 다른 모든 대안 조건일 때 "약품 추가 주입 시작"을 출력하는 if-else 조건 분기 파이썬 코드를 작성해줘.

3. 실행 결과

약품 추가 주입 시작

3-3. 다중 조건 분기문 (if-elif-else)

1. 개념 설명 및 동작 원리

비교 판정할 수치 등급의 경계 한계선이 3개 이상 다단계로 나누어지는 상황에서는 elif를 연속 추가하여 순차 판정을 시행하고, 마지막 남은 평시 등급 범위는 최종 else로 묶어 처리합니다.

[문법 구조]

```
if 조건식1:
    코드1
elif 조건식2:
    코드2
else:
    나머지코드
```

2. 예제 프롬프트

계측 탁도 변수 turb_val = 0.35가 있습니다. 탁도가 0.5 이상이면 "경보", 0.2 이상 0.5 미만이면 "보통", 0.2 미만인 경우 "우수"를 출력하는 다중 조건 분기(if-elif-else) 파이썬 코드를 작성해줘.

3. 실행 결과

보통

3-4. 리스트 반복 순회 (for)

1. 개념 설명 및 동작 원리

반복문 for는 리스트 같은 시퀀스 데이터 그룹을 대상으로, 내부에 담긴 데이터를 0번 인덱스 원소부터 마지막 원소까지 순서대로 하나씩 자동으로 꺼내며 들어쓰기된 실행 로직을 일괄 반복 구동하는 흐름 제어 장치입니다.

[문법 구조]

```
for 임시변수 in 리스트변수:  
    반복할코드(임시변수활용)
```

2. 예제 프롬프트

용수 관로 점검 관리 구역 리스트 `zones = ['A지구', 'B지구', 'C지구']`가 있습니다. for 반복문을 활용하여 구역명들을 순차적으로 출력하되, 각 구역명 앞에 "[점검 대상] "을 붙여 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
[점검 대상] A지구  
[점검 대상] B지구  
[점검 대상] C지구
```

3-5. 반복문 내부 조건 필터링

1. 개념 설명 및 동작 원리

반복 순회 구문 for 내부에 if 조건 제어문을 이중 중첩시키면, 전체 데이터 그룹을 훑으며 특정 정밀 기준(임계치 위반 등)을 만족하는 이상치 원소들만 즉시 포착해 걸러낼 수 있습니다.

[문법 구조]

```
for 원소 in 리스트:  
    if 원소_조건식:  
        조건통과시_코드
```

2. 예제 프롬프트

정수장 탁도 계측 리스트 turb_list = [0.12, 0.55, 0.23, 0.81, 0.19]가 있습니다. for문으로 각 수치를 돌며, if문을 통해 기준치 0.3 NTU 이상인 탁도 수치만 필터링하여 "기준 초과 탁도 검출: 0.00" 형태로 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
기준 초과 탁도 검출: 0.55  
기준 초과 탁도 검출: 0.81
```

3-6. 반복 누적 계산 알고리즘

1. 개념 설명 및 동작 원리

반복 순회 중 생성되는 수치들을 총 누계액에 계속해서 누적 합산하고자 할 때 사용하는 정형 알고리즘입니다. 루프 시작 전 반드시 외부 누적합 보관 변수를 0 등으로 초기화 선언하고, 내부에서 더하기 대입 연산자(+=)로 연쇄 가산합니다.

[문법 구조]

```
누적합변수 = 0
for 값 in 리스트:
    누적합변수 += 값
```

2. 예제 프롬프트

일별 용수 배출 유량 데이터 리스트 `flows = [1200, 1350, 1100]`이 있습니다. 루프 외부에 누적 변수 `total_flow = 0`을 지정하고, `for`문을 통해 리스트 원소들을 돌며 `flows` 데이터를 누계 가산하여 최종 총 용수 공급량을 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

총 용수 공급량: 3650

3-7. 딕셔너리 키-값 쌍 반복 순회 (items)

1. 개념 설명 및 동작 원리

딕셔너리 데이터를 대상으로 반복문을 돌릴 때 `.items()` 메서드를 호출하면, 매 루프 바퀴마다 이름표에 해당하는 키(Key) 명칭과 매핑된 실제 값(Value)이 분해되어 두 개의 반복 제어 변수에 동시에 나누어 매핑 적재됩니다.

[문법 구조]

```
for 키변수, 값변수 in 딕셔너리.items():  
    출력코드(키변수, 값변수 활용)
```

2. 예제 프롬프트

정수장과 실시간 수온 정보가 매핑된 딕셔너리 `temp_data = {"용담": 18.5, "보령": 19.2}`가 주어졌습니다. `.items()` 메서드와 `for` 반복문을 적용하여 딕셔너리를 돌며 "용담정수장 수온: 18.5도", "보령정수장 수온: 19.2도" 포맷으로 화면에 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
용담정수장 수온: 18.5도  
보령정수장 수온: 19.2도
```

3-8. 반복 루프 내 수집용 리스트 필터링

1. 개념 설명 및 동작 원리

대량의 계측 정보 중 특정 조건에 합격한 알짜 데이터들만 모아 새로운 목록을 구축하고자 할 때 사용되는 기법입니다. 루프 바깥에 빈 리스트 []를 선언한 뒤, 내부에서 판정을 만족하는 원소만 .append() 메서드로 누적합니다.

[문법 구조]

```
필터리스트 = []  
for 원소 in 리스트:  
    if 조건식:  
        필터리스트.append(원소)
```

2. 예제 프롬프트

계측 수온 리스트 `temp_list = [15.2, 22.1, 18.0, 24.5, 17.1]`이 주어집니다. 수온이 20도 이상인 고온 계측값만 필터링하여 보관할 빈 리스트 `high_temp_list`를 생성하고, `for`와 `if`를 활용해 20도 이상의 수치만 `high_temp_list`에 적재하여 출력하는 코드를 작성해줘.

3. 실행 결과

고온 수온 리스트: [22.1, 24.5]

3-9. 반복 흐름 제어 (continue)

1. 개념 설명 및 동작 원리

반복 루프 연산 실행 도중 특이 예외 상황(예: 계측기 먹통 에러 데이터 등)을 조우했을 때, 이후에 적힌 복잡한 하위 처리 로직을 가동하지 않고 즉각 다음 순회 원소(Next Loop)로 점프 이전하고자 할 때 continue 문을 실행합니다.

[문법 구조]

```
for 원소 in 리스트:
    if 오류조건:
        continue
    정상가동코드
```

2. 예제 프롬프트

센서 오류 값을 포함한 수온 데이터 리스트 `sensor_data = [15.5, -99.9, 16.2, 999.0, 15.8]`이 있습니다. 수온이 0 미만이거나(음수 오류) 100 이상인(비정상 고온) 값은 계측기 오작동으로 취급해 continue로 건너뛰고, 정상적인 범위 내의 값만 "정상 온도 수신: O.O도" 형태로 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
정상 온도 수신: 15.5도
정상 온도 수신: 16.2도
정상 온도 수신: 15.8도
```

3-10. 중첩 구조 반복 필터링

1. 개념 설명 및 동작 원리

리스트 내부 원소들이 딕셔너리로 저장되어 있는 테이블 데이터 구조에서, for 반복문으로 행을 돌며 각 딕셔너리의 특정 키 속성값(예: 'ph')을 조건 검사하여 조건을 만족하는 대상 명칭을 발췌해 냅니다.

[문법 구조]

```
for 딕셔너리 in 리스트:
    if 딕셔너리['키'] > 임계치:
        print(딕셔너리['이름키'])
```

2. 예제 프롬프트

정수장 DB 리스트 `plant_db = [{"name": "용담", "ph": 7.2}, {"name": "보령", "ph": 5.8}, {"name": "주암", "ph": 8.1}]`가 있습니다. for문으로 순회하며 각 정수장 ph가 6.0 미만이거나 8.0 이상인 (중성 범위를 벗어난) 경우에만 해당 정수장의 이름(name)을 추출하여 "정밀 수질 점검 대상: ○○○ 정수장"으로 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
정밀 수질 점검 대상: 보령정수장
정밀 수질 점검 대상: 주암정수장
```

4. 함수 설계와 예외 처리

4-1. 기본 파라미터 없는 함수 정의 (def)

1. 개념 설명 및 동작 원리

함수(Function)는 중복되는 특정 처리 코드 블록을 하나로 결합하고 고유한 이름을 붙여두어, 필요할 때마다 재호출해 사용하는 재사용 부품입니다. def 함수명():으로 구조화 선언합니다.

[문법 구조]

```
def 함수이름():  
    가동할코드
```

```
# 호출 시  
함수이름()
```

2. 예제 프롬프트

실행되었을 때 콘솔에 "용수 관리 시스템 수질 실시간 모니터링 작동 시작"을 출력해 주는 기본 함수 init_monitoring()을 직접 정의하고 호출하여 결과를 확인하는 파이썬 코드를 작성해줘.

3. 실행 결과

```
용수 관리 시스템 수질 실시간 모니터링 작동 시작
```

4-2. 파라미터(매개변수) 전달 함수

1. 개념 설명 및 동작 원리

함수가 고정된 일만 하지 않고 호출 시점의 입력 상황 데이터에 연동하여 유연하게 동작할 수 있도록, 외부 정보 값을 받아들이는 전용 입력 경로 변수를 괄호 내에 설정한 것을 매개변수(Parameter)라고 칭합니다.

[문법 구조]

```
def 함수명(매개변수1, 매개변수2):  
    동작코드(매개변수활용)
```

2. 예제 프롬프트

정수장 명칭(plant_name)과 수온 데이터(temp) 2가지 정보를 입력받아 "경보: [정수장명] 수온 [수온]도 감지. 미생물 증식 위험 검토 필요." 형식으로 경고문을 포맷 출력하는 report_temp_alert(plant_name, temp) 함수를 구현하고, "용담정수장", 24.5 값을 넘겨 호출하는 코드를 작성해줘.

3. 실행 결과

경보: 용담정수장 수온 24.5도 감지. 미생물 증식 위험 검토 필요.

4-3. 함수 결과물 반환 (return)

1. 개념 설명 및 동작 원리

함수 내부에서 수행한 연산 결과를 메모리 상에서 최종 종결한 뒤, 함수 밖의 대입문이나 변수 호출 지점으로 결과 값을 되돌려 보내 활용하고자 할 때 return 반환할데이터 예약어를 사용합니다.

[문법 구조]

```
def 계산함수(인자):  
    결과 = 인자 * 10  
    return 결과
```

2. 예제 프롬프트

정수장 유입 탁도(in_turb)와 필터 여과 효율(eff, 0~1 사이 실수)을 매개 변수로 받아 최종 여과 후 탁도를 계산하여 (유입 탁도 * (1 - 여과 효율)) 반환하는 calc_filtered_turbidity(in_turb, eff) 함수를 정의하고, 결과값을 final_turb 변수에 담아 출력하는 파이썬 코드를 작성해줘. (in_turb = 0.8, eff = 0.9 전달)

3. 실행 결과

여과 후 최종 예상 탁도: 0.08

4-4. 조건부 불리언(Boolean) 반환 함수

1. 개념 설명 및 동작 원리

입력받은 계측 지표들이 정해진 안전 조건 규격을 안전하게 충족하는지 여부를 함수 내 조건식으로 검사한 뒤, 최종 통과 여부를 참(True) 또는 거짓(False) 상태값으로 반환하게끔 함수를 설계합니다.

[문법 구조]

```
def 검사함수(수치):  
    if 수치 > 기준:  
        return True  
    return False
```

2. 예제 프롬프트

탁도 수치(turb)와 pH 수치(ph)를 받아서, 탁도가 0.5 미만이고 동시에 pH가 6.0 이상 8.0 이하인 안전 등급 규격이면 True를 반환하고, 그렇지 않으면 False를 반환하는 is_water_safe(turb, ph) 함수를 정의하고 turb=0.3, ph=7.2 데이터로 판정 결과를 테스트하여 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

수질 안전성 합격 여부: True

4-5. 텍스트 파싱 및 복합 구조 반환 함수

1. 개념 설명 및 동작 원리

비정형으로 이어진 수집 원시 텍스트 문자열을 분해하여, 이름표를 갖춘 유의미한 정보 딕셔너리로 조립해 최종 반환하는 텍스트 전처리 전문 함수를 정의해 공용화합니다.

[문법 구조]

```
def 파서함수(텍스트):  
    조각 = 텍스트.split('-')  
    return {  
        '이름': 조각[0],  
        '수치': float(조각[1])  
    }
```

2. 예제 프롬프트

"덕소정수장-0.12"와 같이 하이픈(-)으로 정수장 지명과 탁도가 붙어 들어오는 텍스트를 입력받아, 탁도 부분은 실수형으로 형변환하고 {'name': '덕소정수장', 'turbidity': 0.12} 형태의 딕셔너리로 가공해 반환하는 `parse_sensor_data(text)` 함수를 작성하고 실행 결과를 출력해줘.

3. 실행 결과

구조화된 정수장 딕셔너리: {'name': '덕소정수장', 'turbidity': 0.12}

4-6. 파이썬 예외 발생 상황 확인 (ValueError)

1. 개념 설명 및 동작 원리

파이썬 엔진은 숫자로 번역이 불가능한 한글이나 특수 단위 기호가 포함된 글자(예: "0.12NTU")를 강제로 실수 float() 변환하려 요구받으면, 런타임 오류인 ValueError 예외를 발생시키며 즉시 시스템 실행을 강제 중단시킵니다.

[문법 구조]

```
# float("글자") # 에러 유발 상황
```

2. 예제 프롬프트

문자열 변수 `text = "0.12NTU"`가 있습니다. 이 값을 `float(text)`로 직접 변환을 시도할 때 에러(ValueError)가 강제 유발됨을 실제 코드 부분은 주석으로 막아둔 뒤 상세한 에러 유발 원인을 주석으로 기재해 주는 파이썬 코드를 작성해줘.

3. 실행 결과

단위 문자(NTU)가 포함되어 float 형변환 시 ValueError 예외가 런타임에 발생함을 인지합니다.

4-7. 예외 처리 기본구조 (try-except)

1. 개념 설명 및 동작 원리

에러가 예상되는 취약한 연산 코드를 try: 블록 내부에 넣어 보호막을 치고, 실제 에러가 발생한 시점에 시스템 다운 없이 즉시 except 예외명: 우회 복구 경로로 튕겨나가 작동하게 돕는 안전장치입니다.

[문법 구조]

```
try:
    에러가_우려되는코드
except 예외명:
    우회_복구코드
```

2. 예제 프롬프트

텍스트 `text = "0.12NTU"`의 float 형변환을 안전하게 시도하고자 합니다. try-except문을 사용하여 변환 오류(ValueError) 발생 즉시 예외를 캐치해 "수질 데이터 수질 변환 실패 - 노이즈 데이터 감지!"라는 확인 문구를 출력하고 프로그램을 다운 없이 지속 가동하는 파이썬 코드를 작성해 줘.

3. 실행 결과

수질 데이터 수질 변환 실패 - 노이즈 데이터 감지!

4-8. 예외 에러 메시지 캡처 (as e)

1. 개념 설명 및 동작 원리

except 예외명 as e: 방식을 적용하면, 파이썬 인터프리터가 진단해 던진 상세한 시스템 에러 문자열 정보(KeyError, ValueError 등)를 변수 e로 받아와 콘솔이나 로그 텍스트 파일에 안전 기록할 수 있습니다.

[문법 구조]

```
except ValueError as e:  
    print(f"상세정보: {e}")
```

2. 예제 프롬프트

문자열 `text = "0.12NTU"`의 실수형 변환 시도 중 발생하는 `ValueError` 예외를 포착하되, `as error` 구문으로 시스템 내부 에러 사유 문구를 변수로 받아 콘솔에 함께 로그로 출력하는 파이썬 코드를 작성해줘.

3. 실행 결과

시스템 오류 원인 정보: could not convert string to float: '0.12NTU'

4-9. 예외 발생 시 디폴트값 반환 함수

1. 개념 설명 및 동작 원리

수치 변환 함수 내부에서 데이터 파싱 중 노이즈 예외가 발생하더라도, 호출 지점으로 무관측을 뜻하는 기본 안전 표준값(예: 0.0)을 즉시 return하게끔 try-except 예외 방어 처리를 함수 내부에 일체형으로 캡슐화합니다.

[문법 구조]

```
def 안전변환(값):  
    try:  
        return float(값)  
    except ValueError:  
        return 0.0 # 디폴트값 반환
```

2. 예제 프롬프트

정수장 탁도 문자열을 받아서 실수로 변환하는 `get_clean_turbidity(text)` 함수를 설계하고자 합니다. 함수 내부에 try-except ValueError 구조를 설계하여, 단위 혼용 등으로 에러 발생 시 디폴트 수치인 0.0을 즉시 안전 반환하도록 구성하는 파이썬 코드를 작성해줘.

3. 실행 결과

이상치 유입 시 반환값: 0.0

4-10. 다중 예외 처리 및 최종 실행 (finally)

1. 개념 설명 및 동작 원리

발생할 여러 예외 종류를 다중 except 블록으로 개별 대처하는 것과 더불어, 성공 여부와 에러 발발 유무와 전혀 관계없이 무조건 마감 실행되는 공용 마감 영역인 finally: 구문을 사용하여 잔여 자원을 릴리즈합니다.

[문법 구조]

```
try:
    가동코드
except 예외1:
    대처1
finally:
    무조건실행코드 # 리소스 닫기 등
```

2. 예제 프롬프트

try-except-finally 구조를 설계하고자 합니다. try 블록에서 특정 데이터 변환 에러(ValueError) 발생 시의 예외 대처와 함께, 중간 에러 여부와 무관하게 맨 마지막 시점에 "수질 데이터 소켓 통신 채널 안전하게 종료"를 반드시 수행하여 마감하는 파이썬 코드를 작성해줘.

3. 실행 결과

수질 데이터 수치 변환 실패 - 노이즈 데이터 감지!
수질 데이터 소켓 통신 채널 안전하게 종료

[프로젝트 ①] 수자원 센서 원시 데이터 정제기

상황 부여: K-water 관제 시스템에 수질 센서 데이터가 "20231015,정상,팔당댐,0.8"처럼 쉼표로 연결된 긴 텍스트로 유입됩니다. 이 원시 데이터를 정제하여 분석 가능한 데이터 목록으로 만드는 작업을 단계별로 진행합니다.

프로젝트 ① - 1단계: 단일 문자열 분리하기

프롬프트 입력해보기:

"K-water 센서에서 '20231015,정상,팔당댐,0.8'이라는 원시 문자열 데이터가 한 줄로 들어왔어.

1. 이 한 줄짜리 글자 데이터를 쉼표(,)를 기준으로 각각 날짜, 상태, 댐 이름, 탁도라는 4가지 조각으로 분리해 줘.
2. 분리해낸 값들을 화면에 리스트 형태로 출력해 주는 코드를 작성해 줘."

AI가 짜준 코드 실행해보기:

```
raw_text = "20231015,정상,팔당댐,0.8"
parts = raw_text.split(',')
print(parts)
```

프로젝트 ① - 2단계: 분리한 값들을 이름표로 묶기



프롬프트 입력해보기:

"방금 쪼갠 데이터를 분석하기 편하게 정리하자."

1. 쪼갠 값들을 각각 '날짜', '상태', '댐 이름', '탁도'라는 이름표(Key)를 붙여서 하나의 세트 데이터(딕셔너리)로 묶어줘.
2. 이때 탁도 수치는 나중에 연산(크기 비교)이 가능하도록 소수점이 있는 숫자로 형변환을 해 줘.
3. 마지막으로 이 완성된 세트 데이터를 화면에 출력해 줘."

AI가 짜준 코드 실행해보기:

```
raw_text = "20231015,정상,팔당댐,0.8"
parts = raw_text.split(',')
dam_dict = {
    "날짜": parts[0],
    "상태": parts[1],
    "댐 이름": parts[2],
    "탁도": float(parts[3])
}
print(dam_dict)
```

프로젝트 ① - 3단계: 여러 데이터 일괄 처리하기

프롬프트 입력해보기:

"이번엔 대량의 데이터를 일괄 처리해 볼게."

1. 여러 개의 센서 데이터가 담긴 목록(raw_list = ["20231015,정상,팔당댐,0.8", "20231015,정상,소양강댐,1.2"])을 만들어 줘.
2. 목록에 있는 데이터를 반복문을 사용해 위에서부터 하나씩 꺼내어 확인해 줘.
3. 꺼낸 텍스트들을 방금 만든 분리 및 매핑 로직을 활용해 정제한 뒤, 최종적으로 정제된 결과들을 새 목록에 차곡차곡 모아서 한 번에 출력해 줘."

AI가 짜준 코드 실행해보기:

```
raw_list = [  
    "20231015,정상,팔당댐,0.8",  
    "20231015,정상,소양강댐,1.2"  
]  
clean_data_list = []  
for raw in raw_list:  
    parts = raw.split(',')  
    dam_dict = {  
        "날짜": parts[0],  
        "상태": parts[1],  
        "댐 이름": parts[2],  
        "탁도": float(parts[3])  
    }  
    clean_data_list.append(dam_dict)  
print(clean_data_list)
```

프로젝트 ① - 4단계: 통신 에러 데이터 자동 필터링



프롬프트 입력해보기:

"데이터 정제 중에 발생할 수 있는 시스템 오류 데이터를 걸러내는 안전장치를 추가하자."

1. 목록 데이터를 순회하며 정제하는 과정 중에, 텍스트 안에 'ERROR'라는 특정 글자가 포함된 불량 데이터가 있는지 검사해 줘.
2. 만약 오류 데이터가 발견되면, 정제 로직을 실행하지 말고 무시하고 건너뛰어서 다음 정상 데이터로 넘어가도록 예외 조건을 덧붙여줘."

AI가 짜준 코드 실행해보기:

```
raw_list = [  
    "20231015, 정상, 팔당댐, 0.8",  
    "20231015, ERROR, 통신장애, 0.0",  
    "20231015, 정상, 소양강댐, 1.2"  
]  
  
clean_data_list = []  
for raw in raw_list:  
    if "ERROR" in raw:  
        print(f"에러 데이터 무시: {raw}")  
        continue  
    parts = raw.split(',')  
    dam_dict = {  
        "날짜": parts[0],  
        "상태": parts[1],  
        "댐 이름": parts[2],  
        "탁도": float(parts[3])  
    }  
    clean_data_list.append(dam_dict)  
print(clean_data_list)
```

프로젝트 ① - 핵심 문법 해설

이론 학습: 에러가 났을 때 스스로 코드를 읽고 고칠 수 있도록, 방금 작성한 코드의 핵심 기초 문법을 정리합니다.

1. 변수 (Variable)

데이터를 담는 상자입니다. `parts = raw_text.split(',')`처럼 등호(=)를 사용해 우변의 데이터를 좌변의 변수에 대입합니다.

2. 자료형 (Data Type)

문자열 (String): 따옴표로 감싸진 텍스트 (예: "팔당담")

실수형 (Float): 소수점이 포함된 숫자 (예: 0.8). 파이썬은 문자형 수식 "0.8"을 계산이 가능한 실수형 0.8로 변환할 때 `float()` 함수를 씁니다.

3. 딕셔너리 (Dictionary)

`{"날짜": "20231015"}`처럼 이름표(Key)와 실제 값(Value)을 짝지어 다수의 데이터를 명확하게 관리하는 파이썬 고유의 자료 구조입니다.

[프로젝트 ②] 실시간 수질 경보 시스템

상황 부여: 다수의 댐에서 실시간으로 정제된 데이터가 쏟아져 들어오고 있습니다. 이 데이터를 자동으로 반복 점검(조회)하고, 위험 상태 발견 시 컴퓨터 메모장에 신규 기록(저장)하며, 점검 전 기존 로그를 비우는(삭제) 실시간 수명 주기 관리 시스템을 만듭니다.

프로젝트 ② - 1단계: 데이터 조회 및 순회 (Read)



프롬프트 입력해보기:

"수질 모니터링을 위한 기초 데이터 점검 흐름을 만들어보자."

1. 댐 3개의 이름과 탁도 수치 데이터를 입력 데이터 목록으로 선언해 줘.
2. 이 목록의 처음부터 마지막 데이터까지 순서대로 하나씩 가져와서 '조회 중: [댐이름] (탁도: [탁도 수치])' 형태로 화면에 출력해 줘."

AI가 짜준 코드 실행해보기:

```
water_data = [  
    {"이름": "팔당댐", "탁도": 0.8},  
    {"이름": "소양강댐", "탁도": 1.5},  
    {"이름": "대청댐", "탁도": 0.5}  
]  
for data in water_data:  
    print(f"조회 중: {data['이름']} (탁도: {data['탁도']})")
```

프로젝트 ② - 2단계: 실시간 순찰 시간 지연 기능 추가



프롬프트 입력해보기:

"점검 시스템에 실제 실시간 감시 느낌이 나도록 타이밍을 조정해 볼게.

1. 목록에서 데이터를 한 줄씩 꺼내어 화면에 출력할 때마다 동작을 잠깐 멈추게 해줘.
2. 1초의 시간 간격(딜레이)을 두고 천천히 하나씩 출력되도록 기능을 보강해 줘."

AI가 짜준 코드 실행해보기:

```
import time
for data in water_data:
    print(f"조회 중: {data['이름']} (탁도: {data['탁도']})")
    time.sleep(1)
```

프로젝트 ② - 3단계: 위험 수치 발견 시 화면 알림 (신규)



프롬프트 입력해보기:

"수질 데이터의 정상 여부를 감지해 경보를 올려보자."

1. 실시간으로 데이터를 한 줄씩 꺼내는 과정 중에, 탁도 수치가 1.0을 넘어서는 경보 대상 댐이 있는지 검사해 줘.
2. 탁도가 1.0을 초과하면 화면에 즉시 '[경고] [댐이름] 탁도 초과!'라고 위험 알림 문구를 출력하고, 1.0 이하인 정상 댐은 '[댐이름] 정상'이라고 출력하도록 조건문을 덧붙여줘."

AI가 짜준 코드 실행해보기:

```
import time
for data in water_data:
    if data["탁도"] > 1.0:
        print(f"[경고] {data['이름']} 탁도 초과! (수치: {data['탁도']})")
    else:
        print(f"{data['이름']} 정상")
    time.sleep(1)
```

프로젝트 ② - 4단계: 경고 내역 파일에 누적 기록 (저장)



프롬프트 입력해보기:

"위험 상황이 발생했을 때 화면에만 띄우지 않고 기록을 파일로 저장하자.

1. 탁도가 1.0을 초과하여 경고 알림이 뜰 때마다, 그 경고 메시지를 컴퓨터 메모장과 같은 'alert_log.txt' 파일에 기록해 줘.
2. 이전에 작성된 기록이 지워지지 않고, 새로운 경고가 발생할 때마다 파일 끝에 한 줄씩 누적해서 이어적어 저장하도록 기능을 덧붙여줘."

AI가 짜준 코드 실행해보기:

```
import time
for data in water_data:
    if data["탁도"] > 1.0:
        log_msg = f"[경고] {data['이름']} 탁도 초과! (수치: {data['탁도']})\n"
        print(log_msg.strip())
        with open("alert_log.txt", "a", encoding="utf-8") as f:
            f.write(log_msg)
    else:
        print(f"{data['이름']} 정상")
        time.sleep(1)
```

프로젝트 ② - 5단계: 이전 경고 파일 자동 비우기 (삭제)



나의 지시(Prompt):

"마지막으로 수질 점검을 완벽히 초기화하는 안전장치를 추가하자.

1. 새로운 점검을 시작하기 직전, 컴퓨터에 남아있던 이전의 낡은 경고 파일('alert_log.txt')이 있는지 확인해 줘.
2. 파일이 있다면 완전히 깨끗이 지워버려서(삭제해서) 이전 기록이 섞이지 않게 예외 처리를 해줘. 이 안전장치가 포함된 전체 최종 완성 코드를 작성해 줘."

AI가 짜준 코드 실행해보기:

```
import os
import time

if os.path.exists("alert_log.txt"):
    os.remove("alert_log.txt")
    print("과거 로그 파일 삭제 완료.")

water_data = [
    {"이름": "팔당댐", "탁도": 0.8},
    {"이름": "소양강댐", "탁도": 1.5},
    {"이름": "대청댐", "탁도": 0.5}
]
for data in water_data:
    if data["탁도"] > 1.0:
        log_msg = f"[경고] {data['이름']} 탁도 초과!\n"
        print(log_msg.strip())
        with open("alert_log.txt", "a", encoding="utf-8") as f:
            f.write(log_msg)
    else:
        print(f"{data['이름']} 정상")
        time.sleep(1)
```

프로젝트 ② - 핵심 문법 해설 1: 제어문

이론 학습: 데이터 흐름을 자유자재로 다루기 위해 꼭 알아야 하는 조건문과 반복문 구조를 마스터합니다.

1. 조건문 (if-else)

컴퓨터에게 조건 판단 능력을 주는 구조입니다.

`if` 우측의 조건식(예: `탁도 > 1.0`)이 참(True)이면 아래 들여쓰기된 구문을 실행하고, 거짓(False)이면 `else` 구문을 실행합니다.

2. 반복문 (for)

동일한 작업을 자동으로 반복 수행해 주는 구조입니다.

`for data in water_data:` 구문은 `water_data` 리스트 안에 든 알맹이들을 첫 번째부터 하나씩 꺼내 `data`라는 변수에 대입하며 루프를 돕니다.

객체지향 및 실무 GUI 시뮬레이터

과정 개요: 객체지향의 근본 개념을 다진 후, 사이드바 화면 라우팅 구조를 갖춘 데스크톱 댐 관제 시뮬레이터 프로그램을 완성합니다.

프로젝트 ② - 핵심 문법 해설 2: 파일 제어와 딜레이



이론 학습: PC 저장장치에 텍스트를 기록하고 시간 간격을 두는 파이썬 외부 라이브러리 연동 원리입니다.

1. open()과 with open() 구문

파이썬으로 메모장 파일을 제어하는 함수입니다.

"a" (Append) 모드는 기존 메모장 글 아래에 새로운 글씨를 이어쓰기 하겠다는 설정입니다.

`with open(...)` 방식을 쓰면, 코드 블록이 끝날 때 메모장 파일을 열어둔 채 방치하지 않고 알아서 닫아주는(Close) 안전장치가 작동합니다.

2. time.sleep()과 os 모듈

`import time`을 사용해 파이썬에게 초 단위 멈춤 명령 (`time.sleep(초)`)을 지시할 수 있습니다.

`import os` 모듈을 연동하면 컴퓨터에 저장된 파일(alert_log.txt)을 검사하거나 삭제하는 등 운영체제 제어가 가능해집니다.

객체지향 프로그래밍(OOP) 기초

핵심 개념: 프로그램의 크기가 커지면 변수와 함수가 엉켜서 엉망이 됩니다. 이를 방지하기 위해 데이터와 동작을 하나로 묶어 관리하는 '객체지향'을 배웁니다.

1. 클래스(Class)와 객체(Object)의 이해

클래스 (Class)

붕어빵을 구워내는 '붕어빵 틀'이자 프로그램 설계도입니다.

객체 (Object / Instance)

설계도(클래스)를 바탕으로 컴퓨터 메모리에 찍어낸 실제 실체(완성된 붕어빵)입니다.

클래스 설계 및 객체 생성 실습

실습 환경: 설계도 하나만 있으면 서로 다른 데이터를 가진 수십 개, 수백 개의 댐이나 모터를 무한대로 복제해 낼 수 있는 강력함을 확인해 봅니다.

```
# 1단계: 붕어빵 틀(클래스) 설계
class WaterPipe:
    def __init__(self, location):
        self.location = location
        self.status = "정상"

# 2단계: 서로 다른 데이터를 가진 실제 객체 2개 생성
pipe_1 = WaterPipe("서울")
pipe_2 = WaterPipe("부산")
print(pipe_1.location, pipe_1.status)
print(pipe_2.location, pipe_2.status)
```

[프로젝트 ③] 전국 댐 관제 모니터 뼈대 만들기

상황 부여: 텍스트 콘솔창을 벗어나 실제 관제 센터에서 쓸법한 가로 700, 세로 400 크기의 데스크톱 프로그램 창을 띄웁니다. 좌측 메뉴를 클릭할 때마다 우측의 화면이 톱니바퀴처럼 연동되어 전환되는 라우팅 기능을 단계별로 빌드업합니다.

(이 단계부터는 주피터 노트북 대신, VS Code 등에서 실물 파이썬 파일 `app.py`를 만들어 작성하고 실행합니다)

프로젝트 ③ - 1단계: 기본 윈도우 창 띄우기

프롬프트 입력해보기:

"파이썬으로 데스크톱 프로그램을 만드려고 해.

1. 제목이 'K-water 전국 댐 관제 센터'인 가로 700, 세로 400 크기의 데스크톱 창을 띄워줘.
2. 이 창은 나중에 재사용과 확장이 쉽도록 객체지향(클래스) 구조의 파이썬 코드로 설계해 줘."

AI가 짜준 코드 실행해보기:

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("K-water 전국 댐 관제 센터")
        self.resize(700, 400)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

프로젝트 ③ - 2단계: 화면 좌우 분할 및 크기 고정

프롬프트 입력해보기:

"방금 만든 창의 내부 공간을 구획화하자.

1. 화면 전체 레이아웃을 가로로 양분해 줘.
2. 왼쪽 영역은 가로폭을 딱 150으로 고정해 주고, 오른쪽 영역은 남은 공간을 다 쓰도록 레이아웃을 쪼개줘."

(레이아웃 나누기가 적용된 모습은 아래 메뉴 추가 단계에서 확인합니다)

프로젝트 ③ - 3단계: 좌측 영역에 메뉴판 목록 추가



프롬프트 입력해보기:

"좌측 영역에 사용자가 선택할 메뉴를 넣자."

1. 가로폭 150으로 나눈 좌측 영역 안에 위에서 아래로 마우스로 선택할 수 있는 '수위 관리', '방류 기록', '환경 설정' 세 가지 메뉴판 목록을 생성해 줘."

AI가 짜준 코드 눈으로 확인하기:

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget, QHBoxLayout, QListWidget

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("K-water 전국 댐 관제 센터")
        self.resize(700, 400)
        main_layout = QHBoxLayout()
        self.sidebar = QListWidget()
        self.sidebar.setFixedWidth(150)
        self.sidebar.addItem("수위 관리")
        self.sidebar.addItem("방류 기록")
        self.sidebar.addItem("환경 설정")
        main_layout.addWidget(self.sidebar)
        self.setLayout(main_layout)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

프로젝트 ③ - 4단계: 우측 영역에 3개의 겹쳐진 화면 배치



프롬프트 입력해보기:

"우측 영역에 메뉴별로 전환될 화면들을 준비하자.

1. 우측 영역에 '수위 관리 화면', '방류 기록 화면', '환경 설정 화면'이라는 세 페이지를 같은 공간에 겹쳐서 배치해 줘.
2. 각 페이지 화면의 정중앙에는 구분할 수 있는 확인용 글씨만 크게 적어두고 일단 겹쳐만 뒀."

(겹쳐놓은 화면들은 아래 화면 전환 연동 단계에서 최종 확인합니다)

프로젝트 ③ - 5단계: 사이드바 선택에 따른 화면 전환 연동



프롬프트 입력해보기:

"마지막으로 좌측 메뉴 선택과 우측 겹쳐진 화면을 연결하자.

1. 좌측 메뉴판에서 사용자가 마우스로 메뉴 항목을 클릭하여 선택할 때마다, 우측의 겹쳐진 화면도 그에 매칭되는 페이지 화면으로
2. 즉시 전환되어 보이도록 기능을 연동해 줘."

AI가 짜준 코드 실행해보기:

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget, QHBoxLayout, QListWidget, QStackedWidget, QLabel
from PyQt6.QtCore import Qt

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("K-water 전국 댐 관제 센터")
        self.resize(700, 400)
        main_layout = QHBoxLayout()
        self.sidebar = QListWidget()
        self.sidebar.setFixedWidth(150)
        self.sidebar.addItem("수위 관리", "방류 기록", "환경 설정"])
        # 오른쪽 화면 (스택)
        self.stack = QStackedWidget()
```

프로젝트 ③ - 5단계: 사이드바 선택에 따른 화면 전환 연동



```
page1 = QLabel("여기는 '수위 관리' 화면입니다.")
page2 = QLabel("여기는 '방류 기록' 화면입니다.")
page3 = QLabel("여기는 '환경 설정' 화면입니다.")
page1.setAlignment(Qt.AlignmentFlag.AlignCenter)
page2.setAlignment(Qt.AlignmentFlag.AlignCenter)
page3.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.stack.addWidget(page1)
self.stack.addWidget(page2)
self.stack.addWidget(page3)
main_layout.addWidget(self.sidebar)
main_layout.addWidget(self.stack)
self.setLayout(main_layout)
# 메뉴 클릭 시 화면 전환 연결!
self.sidebar.currentRowChanged.connect(self.stack.setCurrentIndex)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

프로젝트 ③ - 핵심 문법 해설: PyQt6 레이아웃

이론 학습: 프로그램 창 내부의 배치 구조를 잡고 화면을 제어하기 위해 반드시 알아야 하는 GUI 전용 설계 개념을 배웁니다.

1. QHBoxLayout (가로 정렬 레이아웃)

자식 위젯이나 다른 레이아웃들을 가로(왼쪽에서 오른쪽)로 차례대로 배치해 주는 수평 상자 형태의 정렬 방식입니다.

2. QListWidget (메뉴판 위젯)

세로 방향으로 목록 항목들을 추가하고, 마우스로 특정 행(Row)을 선택했을 때 '행 번호가 바뀌었다'는 신호를 외부로 던져주는 위젯입니다.

3. QStackedWidget (화면 겹치기 카드 스택)

여러 개의 위젯 화면을 같은 위치에 겹쳐서 올려두고, 특정 인덱스(0, 1, 2...) 번호를 지정하면 해당 번호의 화면만 위로 올려 보여주는 슬라이드 방식의 화면 제어 위젯입니다.

[프로젝트 ④] 멀티 댐 관제 시뮬레이터 완성

상황 부여: 앞서 구축한 사이드바 라우팅 프레임워크 위에, 실제 독립된 댐 데이터를 연동합니다. 수위가 게이지 형태로 깔이며 기록이 누적되고, 위험 시 팝업 경고가 발생하며, 원격 시스템 초기화가 연동되는 관제 시뮬레이터를 완성합니다.

프로젝트 ④ - 1단계: 백엔드 댐 데이터 클래스 설계



프롬프트 입력해보기:

"화면 기능을 만들기 전에, 백엔드에서 댐의 실제 상태 수치를 관리할 댐 데이터 구조 설계도(클래스)를 먼저 만들자.

1. 댐 객체가 처음 생성될 때 댐의 이름(name)과 최대 수위(max_level)를 정하고, 현재 수위는 기본적으로 최대 수위로 딱 차 있게 초기화해 줘.
2. 이 설계도 내부에는 수위를 깎는 '방류 기능'을 넣어줘. 방류를 1번 수행할 때마다 현재 수위가 10씩 줄어들게 하고, 만약 수위가 0 이하로 떨어지면 더 이상 음수로 내려가지 않고 0에 머물게 해 줘.
3. 추가로 수위를 원래 최대 수위로 100% 되돌리는 '리셋 기능'도 포함해 줘. 화면 UI 없이 순수한 클래스 파이썬 코드로만 설계해 줘."

AI가 짜준 코드 눈으로 확인하기:

```
class Dam:
    def __init__(self, name, max_level):
        self.name = name
        self.max_level = max_level
        self.current_level = max_level

    def release_water(self):
        self.current_level -= 10
        if self.current_level < 0:
            self.current_level = 0

    def reset_level(self):
        self.current_level = self.max_level
```

프로젝트 ④ - 2단계: 수위 관리 화면에 게이지 시각화 표시



프롬프트 입력해보기:

"아까 만든 사이드바 프로그램과 방금 설계한 댐 설계도를 합쳐줘. 창 내부에서 소양강댐(최대수위 200)과 팔당댐(최대수위 150)의 객체 2개를 독립적으로 생성하고, 우측의 첫 번째 화면인 '수위 관리' 페이지 좌우에 각 댐의 이름과 현재 수위를 보여주는 막대 그래프 모양 진행바를 표시해 줘."

프로젝트 ④ - 3단계: 방류 버튼, 게이지 실시간 감소 연동



프롬프트 입력해보기:

"방금 만든 수위 관리 화면에 각 댐마다 '방류' 버튼을 추가해 줘. 방류 버튼을 누르면 해당 댐의 수위 데이터가 10 줄어들면서, 동시에 화면에 표시된 막대 그래프 게이지 수치도 실시간으로 10 감소하게 기능을 연결해 줘."

프로젝트 ④ - 4단계: 위험 경고창 팝업 연동



프롬프트 입력해보기:

"수위 관리 화면에서 방류 버튼을 눌러 수위가 최대 수위의 30% 이하로 떨어지면, 화면 중앙에 '경고: 수위 위험!'이라는 경고 알림 팝업창을 띄우는 예외 경보 조건을 덧붙여줘."

프로젝트 ④ - 5단계 & 6단계: 방류 기록 누적 연동



프롬프트 입력해보기:

"이제 두 번째 메뉴인 '방류 기록' 화면의 기능을 구현할게.

1. 두 번째 페이지의 화면 중앙에 방류 내역 기록들이 계속 누적되어 남을 수 있는 넓은 텍스트 글상자를 하나 크게 배치해 줘. (이 글상자는 사용자가 직접 타자를 칠 수 없게 읽기 전용으로 설정해 줘.)
2. 첫 번째 화면에서 소양강댐이나 팔당댐의 방류 버튼을 누를 때마다, 현재 시스템 시간과 함께 '몇 시 몇 분 몇 초에 어떤 댐이 방류되어 현재 수위가 얼마 남았는지'의 텍스트 메시지를 이 방류 기록 화면의 글상자에 한 줄씩 아래로 계속 누적해서 기록하도록 기능을 연동해 줘."

프로젝트 ④ - 7단계: 환경 설정 화면 구현 및 리셋

프롬프트 입력해보기:

"마지막으로 세 번째 메뉴인 '환경 설정' 화면과 리셋 로직을 추가해서 프로그램을 마스터할게.

1. 세 번째 페이지 화면에 '모든 댐 수위 100% 리셋'이라는 큰 버튼을 하나 달아줘.
2. 이 버튼을 누르면, 내부의 소양강댐과 팔당댐의 수위 데이터가 다시 100% 정상 수치로 되돌아가고, 첫 번째 화면의 두 수위 막대바도 다시 끝까지 꽉 차도록 원상복구 시켜줘.
3. 동시에 두 번째 화면의 방류 기록장에도 '--- 전체 시스템 수위 리셋 완료 ---'라는 기록 텍스트를 남기고, 화면에 '수위 복구가 완료되었습니다'라는 안내 알림 팝업창을 띄우도록 전체 코드를 하나로 완성해 줘."

프로젝트 ④ - 핵심 문법 해설

프로젝트 ④ - 핵심 문법 해설

이론 학습: 시뮬레이터 프로그램의 핵심 메커니즘인 시그널/슬롯 연동 및 객체지향 인스턴스의 다중 관리 원리를 배웁니다.

1. 시그널과 슬롯 (Signal & Slot)

PyQt6에서 동작을 연결하는 핵심 톱니바퀴입니다.

사용자가 버튼을 클릭하는 등의 행동이 일어나면 **신호(Signal)**가 가동되고, 이 신호가 작동할 때 우리가 정의한 특정한 동작 함수인 **슬롯(Slot)**이 맞물려 자동으로 실행됩니다.

2. 다중 인스턴스 (Multiple Instances)

하나의 Dam 클래스(붕어빵 틀)를 정의해 두면, 메모리 상에 self.soyang과 self.paldang이라는 서로 독립된 수위 데이터를 가진 댐 객체(인스턴스)들을 무한히 생성해 관리할 수 있어 코드의 엄청난 중복을 예방합니다.

1일차 종합 요약 (Wrap-Up)

복습: 오늘 우리는 단 8시간 만에 실무 프로젝트를 쪼개어 완성해 나가며 파이썬과 GUI의 핵심을 정복했습니다.



실습 발자취 복습

데이터 정제

유효 표 원시 텍스트 분리 및 매핑, 에러 필터링

경보 자동화

시간 지연, 조건 판단 및 경고 로그 텍스트 파일 저장/
삭제

GUI 프레임워크

데스크톱 창 생성, 사이드바 화면 네비게이션 라우팅
연동

시뮬레이터 완성

댐 설계도 결합, 게이지 갱신, 실시간 로그 누적, 팝업
경고 및 전체 시스템 초기화 제어

내일(2일차)은 이 지식을 응용하여 지도 데이터 연동(folium) 및 실시간 수자원 대시보드 그래프(matplotlib)를 구현해 보겠습니다. 수고하셨습니다!